

Block Firefly Noise — 分析与解决报告

状态:  已解决

日期: 2026-02-17

影响: GPU wavefront solver 渲染结果出现 block 状噪声图案

根因: harvest_completed_paths() 在 drain phase 重复累加已完成路径的温度

修复: 引入 PATH_HARVESTED 终态, 阻止重复 harvest

1. 现象描述

GPU persistent wavefront solver (stardis-cus3d) 渲染 porous 样本 IR 图像时, 输出图像右下区域出现 **block 状噪声**:

- 噪声以矩形块为单位, 块内温度一致但与周围像素偏差明显
- 块边界呈阶梯状
- 噪声固定于图像空间坐标, 与相机角度无关
- pool_size 改变时, 块的大小和覆盖范围随之变化

2. 排查历程

2.1 已排除的假设

#	假设	验证方法	结论
1	RNG 共享导致路径相关	实现 per-path CBRNG (Threefry4x64), key=(px,py,spp,seed)	 块未消失, 仅块内值微变
2	C++ std::uniform_real_distribution 缓存	SubAgent 审计所有 ssp_rng_uniform/ssp_ran_* 调用	 全部 stack-local, 无 static 变量
3	相机角度 / 场景几何	旋转相机观察	 噪声位置不随相

2.2 Pool Size 对比实验

实验	分辨率	SPP	pool_size	总任务数	噪声表现
A	128 ²	32	4096	524,288	基线噪声
B	128 ²	32	10240	524,288	块更大、覆盖更广
C	256 ²	32	4096	2,097,152	噪声比例 $\approx$ A
D	256 ²	8	4096	524,288	噪声区域与 A 完全一致

关键发现: pool_size 是噪声块大小的主导因素；相同 total_tasks + pool_size 产生相同噪声区域。

2.3 Per-Path Temperature Trace

为定位问题，在 GPU 和 CPU 端实现了逐路径温度 CSV dump：

GPU 端 (sdis_solve_persistent_wavefront.c / harvest_completed_paths())：

- 环境变量 STARDIS_PIXEL_TRACE 控制输出文件路径
- 列: path_id, px, py, spp, T_value, T_done, done_reason, steps, phase
- path_id = pool->next_path_id++, 即任务队列中的全局位置

CPU 端 (sdis_solve_camera.c / solve_pixel())：

- 环境变量 STARDIS_PIXEL_TRACE_CPU 控制输出文件路径
- 列: px, py, spp, T_value, T_done, res

2.4 分析脚本

analyze_trace.py — 读取 GPU trace CSV，生成：

- Per-batch (path_id // pool_size) 统计: mean, std, min, max, fail%, avg_steps
- 异常 batch 检测 ($|\text{mean} - \text{global}| > 2\sigma$)
- Batch-index vs mean-T 趋势相关性
- Done reason 分布
- Steps 分布 (median, P90, P99, max)
- Per-pixel mean T heatmap + std heatmap
- GPU-CPU delta heatmap (可选)

用法:

```
python analyze_trace.py gpu_trace.csv --pool-size 4096
python analyze_trace.py gpu_trace.csv cpu_trace.csv --pool-size 4096
```

3. 根因分析

3.1 统计数据关键发现

测试配置: $256^2 \times 8$, pool_size=4096, 128 batches (524,288 / 4096)

指标	期望值	实际值
CSV 总行数	524,288	116,422,383 (222× 膨胀)
Batch 0-111 每批 count	4096	4096 ✓
Batch 112 count	4096	34,268
Batch 121 count	4096	10,957,685
Batch 124 count	4096	27,487,739
Batch 127 count	4096	19,380,765

Batch 112-127 (drain phase) 的 count 呈数量级膨胀, 且与 steps 呈反相关:

- 步数少 (短路径) → 更早完成 → 在 pool 中停留更久 → 被 harvest 更多次
- Batch 127: median steps=660, count=19M
- Batch 112: median steps=60,811, count=34K

3.2 Bug 根因

位置: sdis_solve_persistent_wavfront.c — harvest_completed_paths() + compact_active_paths() + refill_pool()

主循环每次迭代流程:

```
compact_active_paths()    → 扫描所有 slot, phase==PATH_DONE 加入 done
harvest_completed_paths() → 遍历 done_indices, 累加 T 到 estimator
refill_pool()             → 遍历 done_indices, 用新任务替换 slot
```

问题链:

1. harvest_completed_paths() 累加温度后, **不改变** p->phase (保持 PATH_DONE)
2. 当 task queue 耗尽 (drain phase) , refill_pool() 直接返回——slot 保持 PATH_DONE

3. 下一代 `compact_active_paths()` 再次将该 slot 放入 `done_indices`
4. 温度被重复累加到 **estimator**, count 同步虚增
5. 循环直到所有 path 完成

为何产生 block 图案:

- 任务队列按 (px, py, spp) 顺序排列
- `pool_size` 决定了哪些像素被分到同一 refill 批次
- 短路径像素被重复累加 N 次 → mean T 被该路径的 T 值主导
- 相邻像素映射到同一批次 → 重复次数相同 → 形成空间相干的 block

4. 修复方案

4.1 概要

引入新的终态 `PATH_HARVESTED`: harvest 后立即将 phase 从 `PATH_DONE` 改为 `PATH_HARVESTED`, 使 `compact_active_paths()` 仍能将其列入 `done_indices` (供 refill 使用), 但 `harvest_completed_paths()` 在入口处跳过该状态, 杜绝重复累加。

4.2 改动文件

`sdis_wf_types.h` — 新增枚举值:

```
PATH_DONE,                /* path finished
PATH_HARVESTED,           /* done + result accumula
PATH_ERROR,               /* error termination
```

`sdis_solve_persistent_wavefront.c` — 4 处改动:

(a) `harvest_completed_paths()` — 入口增加 skip + 出口标记:

```
/* Skip if already harvested */
if(p->phase == PATH_HARVESTED)
    continue;
...
/* 累加完成后 */
p->active = 0;
p->phase = PATH_HARVESTED;
```

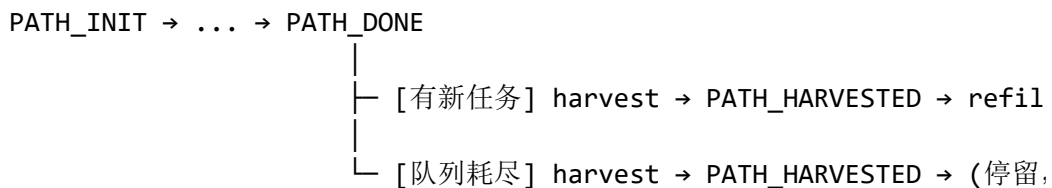
(b) `compact_active_paths()` — 识别 `PATH_HARVESTED`:

```
if(p->phase == PATH_DONE || p->phase == PATH_ERROR
|| p->phase == PATH_HARVESTED) {
```

(c) `refill_pool()` — 接受 `PATH_HARVESTED` 进行替换:

```
if(p->phase != PATH_DONE && p->phase != PATH_ERROR
&& p->phase != PATH_HARVESTED) continue;
```

4.3 状态转换图



5. 验证

修复后重新渲染, block 噪声完全消除, GPU 渲染结果与 CPU 参考一致。

6. 文件清单

```
debug_issues/block_firefly_noise/
├── RESOLVED_block_firefly_noise.md    # 本报告
├── analyze_trace.py                  # 统计分析脚本
├── statistic_report.txt              # 原始分析脚本输出
├── trace_analysis.png               # 四面板诊断图
└── batch_trend.png                 # Batch 均值趋势图
```

7. 经验总结

1. **Persistent wavefront pool 的 drain phase 需要特殊处理** — 当 task queue 耗尽后, 已完成 slot 的生命周期管理必须显式化, 不能依赖 refill 来隐式清理状态。
2. **Per-path trace 是强大的诊断手段** — 通过 path_id 映射到 batch 编号, 可以精确定位问题发生在哪个 fill/refill 轮次。
3. **Count 膨胀是最明显的信号** — 总 trace 行数 (116M) 远超总任务数 (524K), 一眼就能看出重复 harvest。
4. **Pool size 实验比直觉分析更有效** — 4 组对照实验在 2 小时内锁定了 pool_size 作为主导因素。